



**UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE SOBRAL
ENGENHARIA DA COMPUTAÇÃO**

**TRABALHO FINAL: SISTEMA BANCÁRIO
NULLBANK
DOCUMENTO COMPLEMENTAR IV: ESPECIFICAÇÕES
TÉCNICAS**

EQUIPE – INTEGRANTES:

Antônio Kíldere Sousa Menezes – Matrícula: 567258
José Arthur Gomes Azevedo – Matrícula: 567419
Daniel Neco Silva – Matrícula: 568270

Disciplina: Banco de Dados - Período 2026.1
Prof. Fernando Rodrigues de Almeida Júnior

Sobral - CE
2026

1 Ambiente do Banco de Dados Relacional

O ecossistema de armazenamento e persistência de dados do NullBank foi projetado com foco em consistência transacional e isolamento lógico, utilizando as seguintes tecnologias:

- **Sistema Gerenciador de Banco de Dados (SGBD):** MySQL Server (versão 5.7 ou superior).
- **Esquema/Banco de Dados Físico:** Equipe567258.
- **Ferramenta CASE e Modelagem:** MySQL Workbench 8.0 CE (utilizada para geração do Diagrama Entidade-Relacionamento e engenharia reversa do arquivo `bd.mwb`).
- **Recursos Avançados de Banco de Dados:**
 - Gatilhos (*Triggers*): `tg_funcionario_insert`, `tg_funcionario_update` e `tg_funcionario_delete` para auditoria salarial de agências; e `tg_atualizar_saldo_transacao` para automatização matemática de saldos e controle estrito de limites.
 - Procedimentos Armazenados (*Stored Procedures*): `sp_executar_transferencia` para execução atômica de transações financeiras com validação de privilégios em nível de banco de dados.
 - Visões (*Views*): `v_contas_gerente` e `v_extrato_completo` para abstração de consultas complexas.
- **Políticas de Segurança Crítica:** Criptografia unidirecional aplicada a todas as senhas de funcionários e clientes na camada física, utilizando a função hash de alta segurança SHA-256.
- **Site oficial para consulta e download:** <https://dev.mysql.com/downloads/workbench/>

2 Linguagem de Programação e Arquitetura de Software

A aplicação de autoatendimento bancário e retaguarda administrativa foi desenvolvida sob os padrões modernos de engenharia de software:

- **Linguagem de Programação:** Python (versão 3.11 ou superior).
- **Padrão Arquitetural:** Arquitetura modular baseada em pacotes e módulos especializados, garantindo alta manutenibilidade, separação de conceitos e reuso de código:
 - `app.py`: Ponto central de inicialização e roteamento dinâmico de abas de acordo com a credencial de autenticação obtida.
 - `config.py`: Módulo encarregado do carregamento, validação e injeção síncrona das variáveis de ambiente.
 - `database.py`: Abstração e gerenciamento do ciclo de vida das conexões ativas com o SGBD MySQL.

- `auth.py`: Centralizador das rotinas de autenticação multiplataforma, hashing securitário (`hashlib.sha256`) e rotinas de sincronização inicial de usuários.
- `styles.py`: Injeção estruturada de CSS customizado na interface do usuário.
- `views/`: Diretório contendo os submódulos isolados de interface visual (`login.py`, `extrato.py`, `transferencia.py`, `gerente.py`, `usuarios.py`, `extrato_atendente.py` e `operacoes.py`).
- **Site oficial para download do interpretador:** <https://www.python.org/downloads/>

3 Drivers de Conexão e Dependências do Ecossistema

Para estabelecer o acoplamento seguro e a comunicação de rede síncrona entre o código interpretado em Python e a porta lógica do servidor MySQL, foram mapeadas as seguintes dependências em `requirements.txt`:

- **Driver Principal / Conector Nativo:** `mysql-connector-python` (Biblioteca puramente escrita em Python homologada pela Oracle para execução de consultas, transações e chamadas de procedimentos armazenados).
- **Plugin de Autenticação Ativo:** `mysql_native_password` (declarado explicitamente na *string* de conexão para garantir a compatibilidade de autenticação criptografada com o container MySQL).
- **Isolamento de Credenciais de Infraestrutura:** Utilização de arquivos ocultos de ambiente (`.env`) para ocultar dados sensíveis de acesso (`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_PORT`), importados em tempo de execução pela biblioteca nativa `os`.
- **Site oficial do driver de conexão:** <https://pypi.org/project/mysql-connector-python>

4 Frameworks e Bibliotecas de Interface Gráfica (UI/UX)

- **Framework de Desenvolvimento Frontend:** Streamlit.
- **Descrição do Framework:** Biblioteca de código aberto baseada em chamadas síncronas que permite transformar scripts Python em interfaces interativas robustas para aplicações de dados em tempo real.
- **Customização de Identidade Visual Premium:** O NullBank utiliza injeção direta de estilos em blocos `<style>` injetados via Markdown (`unsafe_allow_html=True`). Foi desenvolvido um tema escuro dinâmico (`#0F0F1A`) com aplicação de gradientes, estilização de abas interativas, botões polimórficos de alta responsividade com efeitos visuais de *hover*, e *badges* para distinção hierárquica do cargo logado (*Client*, *Atendente*, *Caixa*, *Gerente* ou *Admin*).
- **Site oficial do Framework:** <https://streamlit.io/>

5 Infraestrutura, Containerização e Mecanismo de Deploy

Buscando atingir portabilidade total entre sistemas operacionais (eliminando incompatibilidades locais de ambiente), o projeto adota conceitos estritos de DevOps:

- **Ambiente Containerizado (Docker):** Toda a infraestrutura do sistema foi isolada em containers através de um arquivo `Dockerfile` (para build da imagem da aplicação Streamlit) e um arquivo `docker-compose.yml` para gerenciamento de serviços.
- **Orquestração de Camadas e Rede Virtual:** O ambiente separa de forma isolada e autônoma o container do Banco de Dados (Backend/MySQL) e o container da Interface Gráfica (Frontend/Streamlit), utilizando diretivas de volume para persistência física do banco e redes internas virtuais para acoplamento dinâmico entre os containers.
- **Mecanismo de Implantação de Produção e Deploy:** Para realizar a disponibilização externa e o deploy seguro do sistema bancário sem expor a infraestrutura local a ataques e sem a necessidade de configurações complexas de redes NAT ou abertura de portas vulneráveis no roteador (*Port Forwarding*), foi implementado o **Cloudflare Tunnel**.
- **Segurança de Tráfego:** O Cloudflare Tunnel cria uma ponte de comunicação de saída puramente criptografada entre o container local e as bordas de rede da Cloudflare, aplicando automaticamente proteção contra ataques de negação de serviço (DDoS) e criptografia de ponta a ponta via certificados TLS.
- **Sites oficiais das ferramentas de infraestrutura:** <https://www.docker.com/> e <https://www.cloudflare.com/>